

Backend Testing and Verification Report

Resource Allocation System

Najib Attar

May 2026

1 Introduction

This report summarizes the backend testing and verification phase for the C++ Resource Allocation System. The system supports users, spaces, requests, allocations, request metadata, lifecycle history, rule-based validation, strategy-based allocation, CSV input/output, and configuration-based strategy selection.

The old `doctest`-based tests were no longer compatible with the current architecture because the source code had changed significantly. In particular, the user model became polymorphic, `User` became abstract, rule classes moved to an `evaluate(...)` interface, and request constructors were extended with metadata fields. Therefore, the outdated tests were replaced with a lightweight local test harness under the `tests/` directory.

The purpose of this testing phase was not only to show that normal examples work, but also to test defect-prone cases, regression risks, malformed input, strategy behavior, CSV loading/exporting, and backend execution.

2 Testing Objective

The main objective was to perform defect-oriented backend testing after many architectural changes. The tests were designed to verify both small unit behavior and larger integration behavior.

The testing goals were:

- verify current backend behavior against the actual source code;
- detect regressions caused by architectural refactoring;
- test invalid and malformed data, not only happy paths;
- verify strategy ordering behavior for Greedy and Priority strategies;
- confirm CSV input and output behavior;
- confirm that backend execution still produces allocation and result files;
- identify remaining risks without claiming full mathematical correctness.

3 Test Environment

The tests were executed in a Windows PowerShell environment using g++ with C++17.

Item	Value
Operating environment	Windows PowerShell
Compiler	g++
Language standard	-std=c++17
Test runner	Local lightweight test harness in <code>tests/</code>
External unit test dependency	None
Main test command	<code>powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1</code>

The local test framework is implemented in `tests/test_framework.h`. It provides simple test registration, assertions, pass/fail output, and a summary of total passed and failed tests.

4 Testing Strategy

The testing strategy combines unit testing, integration testing, backend smoke testing, negative testing, and regression testing.

Test type	Purpose
Unit tests	Verify isolated behavior of models, factories, rules, and services.
Integration tests	Verify interaction between CSV loading, request processing, allocation, and CSV export.
Backend smoke test	Compile and run the real backend executable and verify generated output files.
Negative tests	Use malformed, missing, invalid, and conflicting data to try to break the system.
Regression tests	Protect behavior introduced by previous iterations, such as metadata, lifecycle history, strategy selection, and invalid request handling.
Compiler warning check	Compile with stronger warning flags to detect suspicious code.

5 Verified User Roles and Priorities

The current source code defines the following user roles and priorities:

Role	Priority
Student	1
TeachingAssistant	2
Staff	3
Instructor	4
Administrator	5

The `TeachingAssistant` role is implemented in the source tree. It is supported by `UserFactory`, has its own priority and permissions, and appears in `data/users.csv`. This confirms that `TeachingAssistant` is not only a test assumption, but part of the actual system model.

6 Unit Testing

The unit tests verify individual backend components. These include `TimeSlot`, user subclasses, `UserFactory`, `RequestFactory`, rules, availability logic, and strategy factory behavior.

Examples of unit-tested behavior include:

- `TimeSlot` detects overlap on the same day;
- boundary intervals such as 10:00–12:00 and 12:00–14:00 do not overlap;
- invalid time ranges are rejected;
- `UserFactory` creates supported user subclasses correctly;
- unknown user roles are handled safely;
- role priorities and permissions match the current implementation;
- `RequestFactory` creates `OneTimeRequest`, `RecurringRequest`, `ExamRequest`, and `CommitteeMeetingRequest`;
- malformed request time data creates `InvalidRequest`;
- request title, purpose, and lifecycle history are preserved;
- capacity, feature, status, location, role, and availability rules behave correctly.

7 Integration Testing

Integration tests use temporary CSV files rather than relying only on the normal `data/` folder. This makes the tests reproducible and isolates them from manual demo data.

The integration tests verify that:

- users, spaces, and requests can be loaded through `DataController`;
- loaded requests can be processed through `AllocationService`;
- approved and rejected statuses are assigned correctly;
- `allocations.csv` can be exported;
- `request_results.csv` can be exported;
- exported results include title, purpose, requester role, priority, status, rejection reason, and lifecycle history;
- invalid rows are represented as `InvalidRequest` objects rather than causing crashes.

8 System / Backend Smoke Testing

The PowerShell runner also performs a backend smoke test before running the unit and integration test suite.

The smoke test performs the following steps:

1. compile all backend source files with `g++ -std=c++17`;
2. run the generated backend executable;
3. check the backend exit code;
4. verify that `data/allocations.csv` exists and is not empty;
5. verify that `data/request_results.csv` exists and is not empty.

This confirms that the real executable still runs successfully using the current normal data files.

9 Negative and Regression Testing

The test suite includes defect-oriented cases designed to break hidden assumptions.

The added negative and regression cases include:

- missing users, spaces, and requests files;
- empty users, spaces, and requests files;
- unknown user roles;
- unknown space types;
- duplicate request IDs becoming `InvalidRequest`;
- malformed request rows becoming `InvalidRequest`;
- extra request CSV columns being tolerated;
- zero-length time slots being rejected;
- negative and reversed time data being rejected;
- exact capacity passing;
- one-over capacity failing;
- empty feature and building preferences passing;
- one-time external availability conflicts failing;
- recurring external availability conflicts failing;
- recurring internal self-conflicts failing;
- empty configuration values falling back to the Greedy strategy.

10 Test Coverage Matrix

Feature / Module	Covered	Evidence
TimeSlot overlap, boundaries, invalid ranges	Yes	Same-day overlap, different-day non-overlap, half-open boundary behavior, invalid range rejection.
UserFactory and polymorphic users	Yes	Student, TeachingAssistant, Staff, Instructor, and Administrator creation verified.

Feature / Module	Covered	Evidence
TeachingAssistant support	Yes	Class, factory support, role name, priority, permissions, and data presence verified.
User priorities and permissions	Yes	Role priorities and allowed/requested space types are tested.
RequestFactory valid/invalid creation	Yes	Valid request subtypes and invalid malformed requests are tested.
Request metadata and history	Yes	Title, purpose, default metadata, approval, rejection, and lifecycle events are tested.
DataLoader missing/empty/malformed CSVs	Yes	Missing files, empty files, malformed rows, unknown roles, and unknown spaces are tested.
DataController temporary CSV integration	Yes	Temporary users, spaces, requests, processing, and exports are tested.
InvalidRequest behavior	Yes	Unknown request type, malformed row, invalid user, invalid space, and bad time data are tested.
Capacity, Feature, Status, Location, UserRole rules	Yes	Pass and fail behavior for each rule category is tested.
AvailabilityRule conflicts	Yes	One-time and recurring external conflicts are tested.
RuleEngineFacade through AllocationService	Yes	Requests are processed through AllocationService, which uses RuleEngineFacade internally.
Greedy strategy order	Yes	Conflicting requests verify input-order processing.
Priority strategy order and stability	Yes	Descending priority and same-priority stable order are tested.
AllocationStrategyFactory fallback	Yes	Greedy, Priority, uppercase input, empty input, and unknown strategy fallback are tested.
Config loading fallback	Yes	Greedy, Priority, missing key, invalid value, empty value, and missing config are tested.
CSV exports	Yes	Allocations and request results are exported and checked for expected content.
Backend smoke execution	Yes	Real backend executable is compiled and run before the test suite.

11 Test Results

The main test command was:

```
powershell -ExecutionPolicy Bypass -File tests\run_tests.ps1
```

The result was:

```
Running 55 test(s)  
Result: 55 passed, 0 failed
```

The runner also compiled and ran the backend smoke executable before running the unit and integration tests.

12 Compiler Warning Check

A compiler warning check was performed using:

```
g++ -std=c++17 -Wall -Wextra
```

No compiler warnings were reported during this check.

13 AddressSanitizer Check

An AddressSanitizer build was attempted using:

```
-fsanitize=address
```

However, AddressSanitizer was not available in the current MinGW environment because the linker could not find `-lasan`. Therefore, memory safety is not tool-verified in this testing phase.

14 Defects and Risks Found

No source-code defect requiring an immediate fix was found during this pass.

However, several risks and design gaps were identified:

- **DataLoader** currently allows duplicate user IDs and duplicate space IDs. This does not crash the system, but it can create ambiguous lookup behavior later.

- `DataController::loadAllData(...)` accepts custom users, spaces, and requests paths, but still hardcodes some auxiliary paths such as user busy slots and request participants. This makes isolated integration testing less clean.
- `TeachingAssistant` is implemented in code and appears in data, but documentation should be checked to make sure this role is consistently described.

These are not immediate test failures, but they are important risks for future maintenance.

15 Remaining Limitations

The current backend test suite is stronger than before, but it is not a proof of full correctness.

Remaining limitations include:

- no `AddressSanitizer` or memory-leak tooling is available in the current environment;
- no fuzz testing is performed;
- no exhaustive mathematical verification of allocation algorithms is performed;
- future multi-room exam allocation edge cases may require additional specialized tests;
- generated CSV files are verified for existence and key content, but not every column is exhaustively validated in every scenario.

16 Summary

This backend testing phase replaced outdated tests with a working local test harness and expanded the suite into a defect-oriented regression test set. The current suite covers core models, factories, rules, strategies, configuration loading, CSV loading, CSV exporting, invalid request behavior, request metadata, lifecycle history, and backend smoke execution.

The final test result was:

55 tests passed, 0 tests failed.

No production source-code defect requiring a fix was found. The backend is ready for a formal testing report and demonstration, while still having clearly identified limitations related to duplicate identifiers, auxiliary path hardcoding, and unavailable memory-safety tooling.